

ARIANE 5 FLIGHT 501

TEAM DETAILS

STUDENT A : TECHNICAL FORENSIC ANALYST

Roll No. MSE2026012

STUDENT B : PROCESS, RISK AND ETHICS ANALYST

Roll No. MSE2026007

TECHNICAL FORENSIC ANALYST

1. Timeline

The Technical Forensic Analyst reconstructs the sequence of technical events that caused the Ariane 5 Flight 501 failure.

- Ariane 5 reused significant software from Ariane 4.
- The reused software contained an alignment function that was not required during Ariane 5's flight.
- The software retained assumptions based on Ariane 4's operating conditions.
- Ariane 5 had a different flight trajectory and significantly higher horizontal velocity during its early flight.
- The Horizontal Bias value increased beyond the range expected by the reused software.
- The software attempted to convert a 64 bit floating point value into a 16 bit signed integer.
- The value was outside the representable range of the 16 bit integer.
- An Ada Operand Error occurred.

- The exception caused the SRI, or Inertial Reference System, to shut down.
- The primary and backup SRI systems were affected by the same software failure.
- The On Board Computer received incorrect information from the failed SRI.
- The information was interpreted as flight data.
- Extreme control commands were generated.
- Ariane 5 deviated from its intended flight path.
- The launcher disintegrated and was destroyed.

2. Technical Failure

The immediate technical failure was a numeric overflow in the SRI software.

The software attempted to convert a value that was too large for a 16 bit signed integer.

This generated an exception.

The exception was not safely handled and resulted in the shutdown of the SRI processor.

The failure then propagated to the flight control system because the On Board Computer received incorrect information and interpreted it as valid flight data.

The important point is that the accident was not simply caused by an integer overflow.

The overflow was the immediate technical trigger within a larger chain of requirements, design, implementation and system failures.

3. Requirements, Design and Implementation Issues

Requirements Issues

- Ariane 4 operating assumptions were reused for Ariane 5 without sufficient revalidation.
- The different trajectory characteristics of Ariane 5 were not adequately considered.
- The expected operating range of the Horizontal Bias variable was therefore incorrect for Ariane 5.
- The software requirements did not adequately account for the new operating environment.

Design Issues

- An alignment function that was unnecessary during Ariane 5's flight remained active.
- The exception handling design allowed the SRI processor to shut down when the exception occurred.
- The backup SRI used essentially the same software as the primary SRI.
- This created vulnerability to a common mode software failure.

Implementation Issues

- The software performed a conversion from a 64 bit floating point value to a 16 bit signed integer.
- The conversion was performed without adequately preventing an out of range value.
- The resulting value exceeded the representable range.
- An Ada Operand Error was generated.

These issues show that the failure was not the responsibility of one programming statement alone. Requirements, design and implementation decisions combined to produce the final failure.

4. SDLC Mapping

The technical root causes can be mapped to different phases of the Software Development Life Cycle.

Requirements Phase

- Incorrect assumptions about the operating range of the reused software.
- Insufficient revalidation of Ariane 4 assumptions for Ariane 5.

Design Phase

- Retention of unnecessary functionality.
- Inadequate exception handling strategy.
- Insufficient consideration of common mode software failures.

Implementation Phase

- Unsafe numeric conversion.
- Inadequate protection against out of range values.

Integration Phase

- Insufficient testing of the complete flight control system.
- Insufficient validation of interactions between the SRI and On Board Computer.

System Verification Phase

- Failure to adequately test the software under realistic Ariane 5 trajectory conditions.

The SDLC mapping shows that the failure could have been detected at several stages before the actual launch.

5. Technical Root Causes

- Incorrect assumptions inherited from Ariane 4.
- Reuse of software without adequately validating its assumptions for Ariane 5.
- Continued execution of an unnecessary alignment function.
- Incorrect assumption about the numerical range of the Horizontal Bias value.
- Unsafe floating point to integer conversion.
- Inadequate exception handling.
- Shutdown of the SRI after the exception.
- Common software failure in both primary and backup SRI systems.
- Incorrect interpretation of diagnostic information as flight data.
- Extreme flight control commands.
- Loss of vehicle control.

PROCESS, RISK AND ETHICS ANALYST

1. Organizational and Process Causes

Overconfidence in Reused Software

- The software had previously worked successfully on Ariane 4.
- This created excessive confidence in reusing the same software for Ariane 5.
- Previous success was treated as evidence that the software was suitable for a different operating environment.

Inadequate Revalidation

- The assumptions embedded in the reused software were not sufficiently revalidated.
- Ariane 5 had different operating conditions from Ariane 4.
- The software reuse process did not adequately challenge whether the old assumptions remained valid.

Retention of Unnecessary Functionality

- The alignment function was not required during Ariane 5's flight.
- It was nevertheless retained because of software commonality.
- This introduced unnecessary functionality into a safety critical system.

Inadequate System Level Process

- Component level qualification created confidence in the SRI.
- The complete flight control system was not sufficiently validated under realistic Ariane 5 conditions.
- The organization therefore failed to identify the interaction between the software failure and the flight control system.

2. Testing and Quality Failures

Insufficient Boundary Testing

- Testing should have examined values close to the limits of the 16 bit integer.
- Testing should also have included values beyond the valid range.
- This could have exposed the numeric overflow before launch.

Insufficient Ariane 5 Trajectory Testing

- Testing should have reproduced Ariane 5's actual operating conditions.
- The resulting Horizontal Bias value should have been examined.
- A realistic simulation could have exposed the problem before the launch.

Insufficient End to End Testing

- Testing individual components was not sufficient.
- The complete chain involving the SRI, On Board Computer and flight control system should have been tested together.
- The response of the complete system to an SRI failure should have been evaluated.

Inadequate Common Mode Failure Testing

- The primary and backup SRI systems contained essentially identical software.
- Both could therefore fail because of the same software defect.
- The testing process should have explicitly considered this common mode failure.

3. Human Factors

Confirmation Bias

- Previous success with Ariane 4 created confidence that the reused software was safe.
- This reduced the likelihood of challenging the old assumptions.

Assumption Persistence

- Assumptions valid for Ariane 4 remained embedded in the software used for Ariane 5.
- The change in operating environment was not given sufficient importance.

Over Reliance on Previous Qualification

- Previous testing and successful operation created confidence.
- This confidence was not sufficiently balanced by testing under Ariane 5 specific conditions.

Inadequate Challenge

- The development process did not sufficiently force engineers and reviewers to question whether the Ariane 4 assumptions were still valid.
- A stronger challenge process could have identified the risk before launch.

4. Management Decisions

Decision to Reuse Existing Software

Reason

- Maintain software commonality.
- Avoid unnecessary modification of previously successful software.

Risk

- Existing assumptions were transferred into a different operational environment.

Decision to Retain Unnecessary Functionality

Reason

- Maintain commonality with Ariane 4.

Risk

- Unnecessary functionality remained capable of producing a failure during flight.

Decision Regarding System Level Testing

Reason

- Confidence in previous equipment level qualification.

Risk

- A system level failure could escape component level testing.

Decision Regarding Redundancy

Reason

- Dependence on the backup SRI provided an additional layer of protection.

Risk

- Both systems contained essentially the same software and could therefore suffer the same systematic failure.

5. Ethics

The most relevant ethical principles for this case are related to avoiding harm, maintaining high quality and protecting the public interest.

Avoid Harm

- Engineers working on safety critical systems have a responsibility to consider foreseeable consequences.
- Significant technical risks should be identified, communicated and escalated.
- Safety should be treated as a primary responsibility.

Strive for High Quality

- Safety critical software requires rigorous requirements analysis, testing and validation.
- Previous success of a software component should not replace validation for a new operating environment.

Comprehensive Evaluation

- Safety critical systems should be evaluated under realistic operating conditions.

- Failure scenarios and boundary conditions should be considered.
- The complete system should be tested rather than relying only on component level qualification.

Public Interest

- Safety should take priority over convenience, cost, schedule or software commonality.
- Reusing software should never eliminate the need for safety validation.

6. The duty to escalate was distributed across the engineering organization.

- Should report unsafe assumptions.
- Should identify potential overflow conditions.
- Should communicate limitations of reused software.
- Should raise concerns regarding exception handling.
- Should identify differences between Ariane 4 and Ariane 5.
- Should ensure that Ariane 5 specific requirements were properly defined.
- Should identify differences in operating conditions.
- Should ensure that system level interactions were properly evaluated.
- Should identify missing boundary tests.
- Should identify insufficient trajectory testing.
- Should identify inadequate end to end testing.
- Should identify common mode failures.
- Should evaluate whether redundancy was actually effective against software failures.
- Should challenge unsafe exception handling.

Who Had the Duty to Escalate?

Software Engineers

Requirements Engineers

System Engineers

Testing and Verification Teams

Safety and Quality Engineers

Project and Engineering Management• Should ensure that unresolved safety risks were formally reviewed.

- Should ensure that safety concerns were not ignored because of software commonality or previous successful operation.

7. Three Preventive Process Controls

Control 1: Mandatory Software Reuse Impact Assessment

Before reusing safety critical software:

- Identify all assumptions in the existing software.
- Identify all input ranges.
- Compare the old and new operating environments.
- Revalidate every assumption.
- Obtain approval from system engineering and safety engineering.

How it could have prevented the failure

This process could have identified that Ariane 5's trajectory could produce a much larger Horizontal Bias value than Ariane 4.

Control 2: Mandatory Realistic Closed Loop System Testing

Safety critical systems should be tested using:

- Representative hardware and software.
- Realistic trajectory data.
- Boundary values.
- Abnormal inputs.
- Exception conditions.
- Redundant component failures.

How it could have prevented the failure

A realistic Ariane 5 simulation could have exposed the numeric overflow before the actual launch.

Control 3: Mandatory Safety Review of Exceptions and Redundancy

Every exception capable of shutting down a safety critical component should undergo formal safety analysis.

The review should consider:

- What happens if the exception occurs during operation?
- Can the backup experience the same failure?
- Can both redundant systems fail from the same software defect?
- Is shutdown safer than graceful degradation?
- Can diagnostic information be interpreted as valid control information?

How it could have prevented the failure

This control could have identified the common mode failure risk in the primary and backup SRI systems.

FINAL COMBINED ROOT CAUSE

The Ariane 5 Flight 501 failure resulted from a combination of technical and organizational failures.

Student A identifies the technical chain involving requirements, design, implementation, software reuse, numeric overflow, exception handling and system failure.

Student B identifies the organizational and process failures involving inadequate requirements revalidation, insufficient testing, human factors, management decisions, ethical responsibilities and preventive controls.

The central lesson from the Ariane 5 failure is that successful software reuse does not guarantee safety in a new system.

Whenever software is reused, its requirements, assumptions, operating environment, input ranges, failure behavior and safety implications must be revalidated.

REFERENCES:

- 1) [https://www.esa.int/Newsroom/Press_Releases/Ariane_501 - Presentation of Inquiry Board report](https://www.esa.int/Newsroom/Press_Releases/Ariane_501_-_Presentation_of_Inquiry_Board_report)
- 2) [https://www.researchgate.net/publication/2637332_The_Ariane_5_Flight_501_Failure - A Case Study in System Engineering for Computing Systems](https://www.researchgate.net/publication/2637332_The_Ariane_5_Flight_501_Failure_-_A_Case_Study_in_System_Engineering_for_Computing_Systems)
- 3) <https://cgi.cse.unsw.edu.au/~cs2111/ClassicalB/PDF/the-ariane-flight-failure.pdf>

GitHub Repo link-

MSE2026012-<https://github.com/mse2026012-Harshitha/lab01-failure-forensics.pdf.git>

MSE2026007-<https://github.com/UrviGupta-mse2026007/ASE-lab01-failure-forensics.git>